

Table des matieres

1. Resume executif	3
2. Securite	4
2.1 Points forts identifie	4
2.2 Vulnerabilites critiques	4
2.3 Vulnerabilites importantes	5
2.4 Ameliorations souhaitees de securite	6
3. Base de donnees et persistance	6
3.1 SQLite : limites pour la production	6
3.2 Performance des requetes	7
3.3 Strategie d'indexation	7
3.4 Strategie de migration	8
4. Architecture API et code	8
4.1 Points forts	8
4.2 Code smells et ameliorations	9
4.3 Gestion des erreurs	9
5. Deploiement et infrastructure	10
5.1 Configuration Docker	10
5.2 Reverse proxy et TLS	10
5.3 Observabilite	11
6. Tests et assurance qualite	11
6.1 Etat actuel des tests	11
6.2 Recommandations de tests	12
7. Feuille de route des recommandations	12

7.1 Phase 1 - Pre-production (2 semaines)	12
7.2 Phase 2 - Stabilisation (4 semaines)	12
7.3 Phase 3 - Maturite (8 semaines)	13
8. Conclusion	14

1. Resume executif

Ce rapport presente les resultats de l'audit end-to-end (E2E) complet de la plateforme ONIT-PNG (Observatoire National Intelligent des Telecommunications), developpee pour l'Autorite de Regulation des Postes et Telecommunications (ARPT) de la Republique de Guinee. L'audit couvre l'ensemble des composants de l'application : infrastructure, securite, architecture API, base de donnees, interface utilisateur et deploiement.

La plateforme ONIT-PNG est une application Next.js 16 utilisant Prisma ORM avec SQLite, NextAuth v4 pour l'authentification, et une architecture RBAC (Role-Based Access Control) avec politique RLS (Row-Level Security) pour le filtrage des donnees par role et par organisation. L'application comprend 15+ routes API, 9 tableaux de bord specialises, un systeme d'import de donnees (CSV/JSON), une API prestataire avec authentification par cle API securisee, et un systeme complet d'audit trail.

L'audit revele que la plateforme est dans un etat globalement satisfaisant avec des fondations architecturales solides. Cependant, plusieurs axes d'amelioration critiques ont ete identifies, notamment en matiere de securite avancee, de performance a l'echelle, de strategie de base de donnees pour la production, et d'observabilite operationnelle. Les recommandations sont classees par priorite : critique (a traiter avant toute mise en production), importante (a planifier rapidement), et souhaitable (ameliorations continues).

Domaine	Statut	Score	Priorite
Securite	Partiel	6/10	Critique
Architecture API	Bon	8/10	Important
Base de donnees	Partiel	5/10	Critique
Authentification	Bon	7/10	Important
RBAC / RLS	Tres bon	9/10	Souhaitable
Deploiement	Partiel	6/10	Critique
Observabilite	Insuffisant	3/10	Critique
Tests	Partiel	5/10	Important
Code qualite	Bon	7/10	Souhaitable

Tableau 1 : Synthese des scores par domaine d'audit

2. Securite

2.1 Points forts identifie

La plateforme dispose de plusieurs mecanismes de securite bien implements qui constituent une base solide. L'authentification utilise NextAuth v4 avec une strategie JWT, des cookies securises (httpOnly, SameSite=Lax, Secure en production), et une validation du secret NEXTAUTH_SECRET au demarrage avec un fallback explicite en developpement. Le hachage des mots de passe utilise bcryptjs avec un cost factor de 12, ce qui est conforme aux bonnes pratiques actuelles.

Le systeme RBAC est particulierement bien concu avec 9 roles hierarchiques (de SUPER_ADMIN a PUBLIC), des permissions granulaires par ressource et action (read/write/admin), et un filtrage RLS (Row-Level Security) qui limite automatiquement l'acces aux donnees en fonction du role et de l'organisation de l'utilisateur. Ce filtrage s'applique de maniere coherente sur toutes les routes API qui manipulent des donnees operateur ou region.

Les routes API utilisent systematiquement la validation Zod pour les entrees, avec des transformations de sanitisation HTML (stripHtml) qui prevenient les attaques XSS. Le rate limiting est implemente sur les endpoints sensibles (alertes, prestataires) avec des limites appropriees. Les cle API prestataires sont hachees en SHA-256 avant stockage, et leur validation se fait par comparaison de hachage, jamais en clair.

2.2 Vulnerabilites critiques

V-CRIT-01 : Secret NEXTAUTH_SECRET expose et fallback faible

Le fichier .env contient le secret NEXTAUTH_SECRET en clair et ce meme secret est utilise comme valeur par default dans docker-compose.yml. En cas d'acces non autorise au depot de code source ou a la configuration Docker, un attaquant peut forger des jetons JWT valides et prendre le controle total de l'application. De plus, le code source contient un fallback hardcoded ('dev-only-secret-do-not-use-in-production') qui, bien qu'accompagne d'un avertissement, represente un risque si l'environnement est mal configure.

Recommandation : Utiliser un gestionnaire de secrets (Vault, AWS Secrets Manager, ou variables d'environnement Docker Swarm/Kubernetes secrets). Supprimer le fallback hardcoded. Le demarrage de l'application doit echouer immediatement si NEXTAUTH_SECRET n'est pas defini en production, sans fallback possible.

V-CRIT-02 : Absence de protection CSRF sur les routes API

Bien que NextAuth fournisse un jeton CSRF pour les routes d'authentification, les autres routes API (POST /api/mesures, POST /api/scores, POST /api/campaigns, PATCH /api/users, etc.) ne verifient pas la presence d'un jeton CSRF. Un attaquant peut concevoir une page web malveillante qui soumet des requetes POST au nom d'un utilisateur authentifie (attaque CSRF), modifiant ainsi des donnees sans le consentement de la victime. Cette vulnerabilite est particulierement critique pour les operations d'ecriture sur les mesures QoS et les scores des operateurs.

Recommandation : Implementer un middleware CSRF qui genere un jeton par session et verifie sa presence dans les en-tetes (X-CSRF-Token) ou dans le corps des requetes POST/PATCH/DELETE. Alternativement, exiger l'en-tete Content-Type: application/json pour toutes les requetes mutantes (ce qui bloque les soumissions de formulaires HTML classiques).

V-CRIT-03 : Absence de verrouillage de compte apres echecs

L'authentification n'implemente aucun mecanisme de verrouillage de compte apres un nombre d'echecs consecutifs. Un attaquant peut effectuer des attaques par force brute ou par dictionnaire sans aucune limitation au niveau du compte. Le rate limiting existant est base sur l'adresse IP, ce qui est insuffisant contre les attaques distribuees ou les attaquants utilisant des proxies rotatifs. De plus, le message d'erreur retourne lors d'un echec de connexion ne divulgue pas d'information sensible, ce qui est positif, mais l'absence de verrouillage reste une faille majeure pour un systeme reglementaire.

Recommandation : Implementer un compteur d'echecs par compte utilisateur avec verrouillage progressif (5 echecs = 15 min, 10 echecs = 1h, 20 echecs = compte desactive). Logger les tentatives echouees dans l'audit trail avec l'adresse IP. Envoyer une notification a l'administrateur en cas de tentative suspecte.

2.3 Vulnerabilites importantes

V-IMP-01 : Rate limiting en memoire uniquement

Le systeme de rate limiting utilise un Map en memoire (Map<string, {count, resetTime}>). Cette approche presente trois problemes majeurs en production. Premierement, les compteurs sont perdus a chaque redemarrage du serveur, permettant a un attaquant de contourner les limites en provoquant des redemarrages. Deuxiemement, dans un deploiement multi-instance (load balancer), chaque instance maintient ses propres compteurs, divisant effectivement les limites par le nombre d'instances. Troisiemement, le nettoyage periodique via setInterval ne garantit pas une liberation immediate de la memoire sous forte charge.

Recommandation : Migrer le rate limiting vers Redis avec une strategy sliding window. Redis offre la persistance, le partage entre instances, et l'expiration automatique des cles. Si Redis n'est pas disponible, utiliser une base de donnees dediee avec un TTL.

V-IMP-02 : Generation de cles API avec Math.random()

La fonction generateApiKey() dans utils-api.ts utilise Math.random() pour generer la partie secrete des cles API. Math.random() n'est pas cryptographiquement sur et produit des sequences previsibles si le seed interne est devinable. Un attaquant determinant pourrait theoriquement generer des cles API valides pour un operateur donne.

Recommandation : Remplacer Math.random() par crypto.randomBytes(32) du module 'crypto' de Node.js, qui utilise le generateur pseudo-aleatoire cryptographique du systeme d'exploitation (CSPRNG).

V-IMP-03 : Absence de headers de securite HTTP

L'application ne configure pas les en-tetes de securite HTTP recommandes : Content-Security-Policy (CSP), Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, et Referrer-Policy. Sans CSP, l'application est vulnérable aux attaques XSS par injection de scripts meme avec la sanitisation Zod. Sans HSTS, les cookies de session peuvent etre interceptes lors d'une attaque man-in-the-middle sur la premiere connexion.

Recommandation : Ajouter un middleware Next.js qui injecte les en-tetes de securite sur toutes les reponses. Configurer CSP pour n'autoriser que les sources de scripts necessaires, HSTS avec un max-age d'au moins 1 an, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, et Referrer-Policy: strict-origin-when-cross-origin.

2.4 Ameliorations souhaitees de securite

V-SOU-01 : Absence d'authentification multi-facteurs (2FA)

Pour une plateforme reglementaire gerant des donnees sensibles de telecommunications, l'authentification a facteur unique represente un risque acceptable uniquement si d'autres mesures compensatoires sont en place. L'ajout d'une authentification TOTP (Time-based One-Time Password) pour les roles admin et super-admin renforcerait significativement la posture de securite. L'implementation peut se faire via la bibliotheque otplib et un mecanisme de verification dans le callback authorize() de NextAuth.

V-SOU-02 : Pas de rotation automatique des cle API

Les cle API prestataires sont statiques et n'ont pas de mecanisme d'expiration ou de rotation. Une cle compromise reste valide indefiniment. Il est recommande d'ajouter une date d'expiration aux cle API (ex: 90 jours) et un mecanisme de rotation sans interruption de service (generer une nouvelle cle pendant que l'ancienne reste valide pendant une periode de grace de 24h).

3. Base de donnees et persistance

3.1 SQLite : limites pour la production

L'application utilise SQLite comme systeme de gestion de base de donnees via Prisma ORM. Si SQLite est parfaitement adapte au developpement et aux petites deployments mono-instance, il presente des limitations structurelles pour un usage en production a l'echelle nationale. SQLite ne supporte qu'un seul writer a la fois, ce qui signifie que toutes les operations d'ecriture sont serialisees. Sous charge concurrente (plusieurs utilisateurs important des donnees simultanement, ou plusieurs prestataires soumettant des mesures), les performances se degradent rapidement avec des contentions de verrous.

De plus, SQLite ne supporte pas les connexions reseau, ce qui rend impossible un deploiement multi-instance avec partage de base de donnees. Le fichier de base de donnees est stocke localement (file:/home/z/my-project/db/custom.db), ce qui cree une dependance forte au systeme de fichiers de la machine hote et complique les sauvegardes, la reprise apres sinistre, et la mise a l'echelle

horizontale.

Critere	SQLite	PostgreSQL
Concurrence ecriture	Serialisee (1 writer)	Multi-writer MVCC
Connexions reseau	Non	Oui (TCP)
Index partiels	Non	Oui
Full-text search	FTS5 (basique)	tsvector (avance)
Scalabilite	Mono-instance	Horizontale
Sauvegarde a chaud	Limitation	WAL streaming

Tableau 2 : Comparaison SQLite vs PostgreSQL pour ONIT-PNG

Recommandation : Migrer vers PostgreSQL pour la production. Prisma ORM facilite cette migration car il abstrait les differences SQL. Le schema Prisma actuel est compatible avec PostgreSQL (les enums, les relations, et les contraintes uniques sont supportees). La migration implique principalement le changement de la chaine de connexion DATABASE_URL et l'execution de `prisma migrate deploy`. Un plan de migration progressif est recommande : maintenir SQLite en developpement et PostgreSQL en staging/production.

3.2 Performance des requetes

L'audit du code API revele un probleme de performance N+1 dans la route du tableau de bord principal (`/api/dashboard/route.ts`). La section calculant les statistiques par region execute une boucle de requetes SQL individuelles pour chaque region (comptage total, comptage couvert, aggregation QoE, comptage zones blanches), generant potentiellement $4 * N$ requetes supplementaires ou N est le nombre de regions (8 en Guinee). Pour un total de 32 requetes supplementaires par appel au tableau de bord, ce qui est sous-optimal.

Recommandation : Remplacer la boucle de requetes individuelles par des requetes d'agregation `groupBy`. Par exemple, utiliser `prisma.measureQoS.groupBy({ by: ['regionId'], _count: true })` pour obtenir les comptes par region en une seule requete. Pour les metriques QoE, utiliser une requete SQL brute avec `GROUP BY` et `AVG()`. Cette optimisation reduirait le nombre de requetes de 32 a 4-5 environ, ameliorant le temps de reponse du dashboard de maniere significative.

3.3 Strategie d'indexation

Le schema Prisma actuel ne definit pas d'index explicites au-dela des cles primaires et des contraintes uniques. Pour les requetes frequentes identifiees dans l'audit, les index suivants sont recommandes pour ameliorer significativement les performances de lecture. Ces index sont particulierement importants lorsque le volume de mesures QoS augmente (des milliers a des centaines de milliers d'enregistrements).

Table	Index	Justification
MesureQoS	operateurId + timestamp	Filtrage principal du dashboard
MesureQoS	regionId + timestamp	Statistiques par region
MesureQoS	campagneId	Liste des mesures par campagne
Alerte	operateurId + isResolved	Comptage alertes actives par operateur
Alerte	type + isResolved + regionId	Zones blanches par region
ScoreOperateur	operateurId + periode	Dernier score par operateur (deja unique)
AuditLog	userId + createdAt	Journal d'audit par utilisateur

Tableau 3 : Index recommandes pour la production

3.4 Strategie de migration

L'application utilise actuellement prisma db push au lieu de prisma migrate dev pour la gestion du schema. La commande db push est concue pour le prototypage rapide et ne maintient pas d'historique des migrations. En production, c'est un risque majeur car il n'y a pas de traconabilite des changements de schema, pas de rollback possible, et pas de garantie que la base de donnees de production est dans un etat coherent avec le code. Pour un systeme reglementaire comme ONIT-PNG, la tracabilite des changements de schema est une exigence fondamentale.

Recommandation : Adopter Prisma Migrate comme strategie de gestion du schema. Creer la migration initiale a partir du schema actuel avec `prisma migrate dev --name init`. Ensuite, chaque modification du schema doit generer une migration avec `prisma migrate dev --name description_du_changement`. En production, utiliser `prisma migrate deploy` (pas dev) pour appliquer les migrations de maniere controlee. Stocker les fichiers de migration dans le depot de code source pour tracabilite.

4. Architecture API et code

4.1 Points forts

L'architecture API est globalement bien concue avec une separation claire des responsabilites. Les routes API suivent un modele coherent : validation des permissions via RBAC, filtrage RLS des donnees, validation Zod des entrees, operations Prisma, et journalisation d'audit. La couche de librairie (`src/lib/`) est bien organisee avec des modules dedies : `rbac.ts` pour les permissions, `error-handler.ts` pour la gestion centralisee des erreurs, `logger.ts` pour la journalisation structuree, `utils-api.ts` pour les utilitaires partages, et `db.ts` pour le singleton PrismaClient.

L'API prestataire (`/api/prestataires/`) merite une mention speciale pour sa conception securisee. Elle utilise une authentication par cle API (`X-API-Key`) avec validation contre un hachage SHA-256 stocke

en base, un rate limiting dedie (30 req/min), une journalisation d'audit specifique aux prestataires, et une validation Zod complete avec sanitisation HTML. Ce modele pourrait servir de reference pour d'autres endpoints externes futurs.

4.2 Code smells et ameliorations

CS-01 : Repetition du cast de session utilisateur

Chaque route API repete le meme pattern de cast : `(session.user as Record<string, unknown>).role`, `(session.user as Record<string, unknown>).id`, etc. Ce pattern est verbeux, sujet aux erreurs de frappe, et difficile a maintenir. Il existe environ 30+ occurrences a travers le codebase.

Recommandation : Creer un fichier `src/lib/session.ts` avec des fonctions typees : `getUserRole(session)`, `getUserId(session)`, `getUserOrg(session)`. Ces fonctions centralisent le cast et garantissent la coherence. Ajouter le type etendu dans `next-auth.d.ts` pour que TypeScript reconnaisse les proprietes etendues du `session.user`.

CS-02 : Duplication de stripHtml

La fonction `stripHtml` (`val.replace(/<[^>]*>/g, '')`) est definie independamment dans au moins 5 fichiers de routes API (alerts, scores, mesures, campaigns, reports) alors qu'elle existe deja dans `utils-api.ts`. Cette duplication cree un risque d'incoherence si un bug est corrige dans une copie mais pas dans les autres.

Recommandation : Supprimer les definitions locales et utiliser l'import depuis `@/lib/utils-api` : `import { stripHtml } from '@lib/utils-api'`.

CS-03 : Typage insuffisant des operations Prisma en masse

Plusieurs operations d'insertion en masse utilisent des casts '`as any`' pour contourner les types Prisma stricts : `db.measureQoS.createMany({ data: chunk as any })`. Ces casts suppriment la verification de type et pourraient masquer des erreurs de structure de donnees.

Recommandation : Definir des types `Prisma.Prisma.MesureQoSCreateManyInput[]` explicites pour les donnees d'insertion en masse. Utiliser les types generes par Prisma plutot que des types ad hoc.

4.3 Gestion des erreurs

La gestion des erreurs est partiellement centralisee. Le module `error-handler.ts` definit des classes d'erreur personnalisees (`AppError`, `ValidationError`, `AuthenticationError`, `AuthorizationError`, `NotFoundError`) et une fonction `handleApiError()` pour formater les reponses d'erreur de maniere coherente. Cependant, la plupart des routes API utilisent encore le pattern `try/catch` avec `console.error()` et `NextResponse.json()` directs, sans passer par `handleApiError()`. Seule la route `/api/dashboard/route.ts` utilise cette fonction centralisee.

Recommandation : Generaliser l'utilisation de `handleApiError()` dans toutes les routes API. Standardiser le format d'erreur de reponse : `{ error: string, code?: string, details?: object }`. Utiliser les classes d'erreur personnalisees dans les routes API pour lever des erreurs specifiques qui seront interceptees et formatees par le handler centralise. Ajouter les codes d'erreur normalises

(VALIDATION_ERROR, AUTHENTICATION_ERROR, etc.) dans toutes les reponses d'erreur.

5. Deploiement et infrastructure

5.1 Configuration Docker

L'application dispose d'un Dockerfile multi-stage bien conçu avec trois étapes : dependencies, builder, et runner. Le runner utilise un utilisateur non-root (nextjs), ce qui est une bonne pratique de sécurité. Le healthcheck vérifie l'endpoint /api/auth/session avec des paramètres raisonnables (30s interval, 10s timeout, 3 retries). Cependant, plusieurs problèmes ont été identifiés dans la configuration de déploiement.

D-CRIT-01 : Secret par défaut dans docker-compose.yml

Le fichier docker-compose.yml contient un secret par défaut hardcodé : NEXTAUTH_SECRET=\${NEXTAUTH_SECRET:-onit-png-secret-key-2026-guinee-docker}. Ce secret est visible dans le dépôt de code source et peut être utilisé en production si la variable d'environnement n'est pas explicitement définie. Combiné avec la vulnérabilité V-CRIT-01, ceci crée un risque de forge de jetons JWT.

Recommandation : Supprimer la valeur par défaut du secret dans docker-compose.yml. Le fichier docker-compose.yml doit échouer au démarrage si NEXTAUTH_SECRET n'est pas défini. Utiliser Docker secrets ou un fichier .env.docker (exclu du dépôt) pour fournir les secrets au conteneur.

D-IMP-01 : Absence de stratégie de sauvegarde de base de données

La base de données SQLite est montée dans un volume Docker (onit-db:/app/db), mais aucun mécanisme de sauvegarde automatique n'est configuré. En cas de corruption du fichier SQLite ou de perte du volume, toutes les données sont perdues définitivement. Pour un système réglementaire, la perte de données de mesures QoS et de journaux d'audit est inacceptable et pourrait avoir des conséquences légales.

Recommandation : Implémenter un cron job de sauvegarde qui : (1) effectue un snapshot SQLite toutes les 6 heures via sqlite3 db .dump, (2) compresse et chiffre le dump, (3) le stocke dans un emplacement externe (S3, stockage objet, serveur de sauvegarde dédié). Tester régulièrement la restauration. Ajouter un script de vérification d'intégrité (PRAGMA integrity_check) dans le healthcheck.

5.2 Reverse proxy et TLS

Le Caddyfile configure un reverse proxy sur le port 81 vers le port 3000 de l'application. La configuration inclut les en-têtes X-Forwarded-For, X-Forwarded-Proto, et X-Real-IP, ce qui est correct. Cependant, la configuration n'utilise pas les capacités de gestion automatique de TLS de Caddy, qui est l'un des principaux avantages de ce serveur. Sans TLS, les communications entre le client et le serveur transitent en clair, exposant les cookies de session et les données sensibles.

Recommandation : Configurer Caddy avec un nom de domaine et activer le HTTPS automatique. Caddy gere automatiquement l'obtention et le renouvellement des certificats Let's Encrypt. Exemple : `onit.arpt.gn { reverse_proxy localhost:3000 }`. Si un domaine n'est pas encore disponible, utiliser au minimum un certificat auto-signé pour le staging et configurer HSTS.

5.3 Observabilite

L'observabilite de l'application est actuellement insuffisante pour un systeme en production. Le logger implemente des niveaux de journalisation (debug, info, warn, error) avec un format structure, mais les logs sont diriges uniquement vers la console. Il n'y a pas de collecte centralisee, pas de metriques d'application (Prometheus, Datadog), pas de tracing distribue, et pas de tableau de bord de monitoring. Pour un systeme national de supervision des telecommunications, l'absence d'observabilite rend le diagnostic des problemes en production extremement difficile.

Recommandation : Implementer une strategie d'observabilite en trois couches. Premierement, ajouter un endpoint `/api/health` qui retourne le statut de l'application, de la base de donnees, et des services dependants. Deuxiemement, integrer un systeme de metriques (compteur de requetes, temps de reponse par endpoint, nombre d'erreurs, nombre d'utilisateurs actifs) expose au format Prometheus. Troisiemement, configurer la journalisation structuree vers un agrégateur de logs (ELK, Loki, ou CloudWatch).

6. Tests et assurance qualite

6.1 Etat actuel des tests

L'application dispose d'une suite de tests E2E Playwright dans le repertoire `e2e/` couvrant les scenarios suivants : authentication, API utilisateurs/roles/scores, API campagnes, API mesures, API alertes, API prestataires, API rapports, navigation UI, carte SIG, et scenarios de production. C'est une base solide qui couvre les principaux parcours utilisateurs et les endpoints API critiques.

Cependant, il n'existe aucun test unitaire pour les fonctions critiques des librairies partagees (`rbac.ts`, `utils-api.ts`, `error-handler.ts`). Les fonctions comme `checkPermission()`, `getAccessibleOperators()`, `validateApiKeySecure()`, et `checkRateLimit()` sont essentielles a la securite de l'application et devraient etre testees unitairement avec des cas limites et des scenarios d'erreur.

Type de test	Existant	Couverture	Recommandation
E2E Playwright	Oui	Moyenne	Etendre les cas limites
Tests unitaires	Non	0%	Priorite haute pour lib/
Tests integration API	Partiel	Faible	Ajouter tests sans UI
Tests de charge	Non	0%	k6 ou Artillery

Tests de securite	Non	0%	OWASP ZAP
-------------------	-----	----	-----------

Tableau 4 : Etat et recommandations des tests

6.2 Recommandations de tests

Priorite 1 - Tests unitaires des librairies critiques : Creer des suites de tests Jest ou Vitest pour rbac.ts (tester chaque role avec ses permissions attendues, tester les acces refuses, tester les cas limites ou les policies n'existent pas), utils-api.ts (tester la validation des cles API, le rate limiting avec differents scenarios de charge, la sanitisation HTML avec des payloads XSS), et error-handler.ts (tester chaque type d'erreur et son format de reponse).

Priorite 2 - Tests d'integration API : Utiliser un setup de test avec une base de donnees SQLite en memoire pour tester les routes API de bout en bout sans dependre de l'interface utilisateur. Ces tests doivent verifier les codes de statut HTTP, les formats de reponse, le filtrage RLS, et les validations Zod avec des donnees invalides.

Priorite 3 - Tests de charge : Utiliser k6 ou Artillery pour simuler un trafic realiste (50 utilisateurs simultanés, 1000 mesures importees en masse, 10 requetes dashboard par seconde). Identifier les goulots d'etirement et les temps de reponse maximaux avant degradation.

7. Feuille de route des recommandations

7.1 Phase 1 - Pre-production (2 semaines)

ID	Recommandation	Priorite	Effort
V-CRIT-01	Rotation secrets NEXTAUTH_SECRET	Critique	1 jour
V-CRIT-02	Protection CSRF sur API	Critique	2 jours
V-CRIT-03	Verrouillage compte echecs	Critique	1 jour
D-CRIT-01	Supprimer secret par default Docker	Critique	0.5 jour
V-IMP-03	Headers securite HTTP	Critique	1 jour
V-IMP-02	Crypto.randomBytes pour API keys	Importante	0.5 jour
D-IMP-01	Script sauvegarde base	Importante	1 jour

Tableau 5 : Phase 1 - Actions critiques pre-production

7.2 Phase 2 - Stabilisation (4 semaines)

ID	Recommandation	Priorite	Effort
DB-01	Migration PostgreSQL	Critique	3 jours
DB-02	Ajout index performance	Importante	1 jour
DB-03	Prisma Migrate	Importante	1 jour
V-IMP-01	Rate limiting Redis	Importante	2 jours
CS-01	Helper session type	Importante	1 jour
CS-02	Dedup stripHtml	Importante	0.5 jour
CS-03	Typage Prisma createMany	Souhaitable	1 jour
TEST-01	Tests unitaires lib/	Importante	3 jours
DEP-01	Endpoint /api/health	Importante	1 jour
DEP-02	TLS Caddy	Importante	1 jour

Tableau 6 : Phase 2 - Stabilisation

7.3 Phase 3 - Maturite (8 semaines)

ID	Recommandation	Priorite	Effort
V-SOU-01	Authentification 2FA (TOTP)	Souhaitable	3 jours
V-SOU-02	Rotation cles API	Souhaitable	2 jours
OBS-01	Metriques Prometheus	Souhaitable	3 jours
OBS-02	Logs structures (ELK/Loki)	Souhaitable	2 jours
TEST-02	Tests integration API	Souhaitable	3 jours
TEST-03	Tests de charge (k6)	Souhaitable	2 jours
TEST-04	Scan securite OWASP ZAP	Souhaitable	2 jours
FEAT-01	Notifications temps reel	Souhaitable	5 jours
FEAT-02	Export PDF/Excel rapports	Souhaitable	3 jours
FEAT-03	Versioning API (/v1/)	Souhaitable	2 jours

Tableau 7 : Phase 3 - Maturite

8. Conclusion

La plateforme ONIT-PNG dispose de fondations solides pour un système de supervision des télécommunications au niveau national. L'architecture RBAC/RLS est particulièrement bien conçue et constitue un atout majeur pour un système réglementaire où la séparation des accès est primordiale. La validation systématique des entrées, la sanitisation HTML, et la journalisation d'audit sont des pratiques exemplaires qui témoignent d'une réflexion approfondie sur la sécurité.

Les recommandations formulées dans ce rapport visent à élever la plateforme au niveau exigé par un déploiement en production pour un organisme de régulation. Les actions critiques (Phase 1) concernent principalement la sécurisation des secrets, la protection CSRF et le verrouillage des comptes, qui sont des prérequis indispensables avant toute mise en production. La Phase 2 adresse les problèmes de performance et de fiabilité avec la migration PostgreSQL et l'observabilité. La Phase 3 prépare l'avenir avec des fonctionnalités avancées comme la 2FA, les métriques, et les notifications temps réel.

L'estimation totale des efforts pour les trois phases est d'environ 50 jours-homme, répartis sur 14 semaines. Cette feuille de route est réaliste et permet une mise en production progressive tout en améliorant continuellement la qualité et la sécurité de la plateforme. Il est recommandé de commencer immédiatement les actions de Phase 1, car elles représentent les risques les plus élevés et les corrections les plus rapides.

Phase	Durée estimée	Effort (j/h)	Jalon
Phase 1 - Pre-production	2 semaines	7 jours	Go/No-Go production
Phase 2 - Stabilisation	4 semaines	18 jours	Production stable
Phase 3 - Maturité	8 semaines	25 jours	Plateforme mature

Tableau 8 : Synthèse des phases et efforts